reptimeline: Tracking Discrete Representation Evolution During Neural Network Training

J. Arturo Ornelas Brand
Independent Researcher
arturoornelas62@gmail.com
https://github.com/arturoornelasb/reptimeline

March 2026

Abstract

We present **reptimeline**, a Python library for tracking how discrete representations evolve during neural network training. Unlike scalar logging tools (WandB, TensorBoard) that report aggregate metrics, reptimeline tracks per-code lifecycle events: when concepts become distinguishable (births), when representations collapse (deaths), when relationships form between concept pairs (connections), and where phase transitions occur in training dynamics. The library additionally discovers what each code element encodes—anti-correlated pairs, dependency chains, and three-way AND-gate interactions—without requiring prior ontological knowledge, with all discoveries subjected to multiple-comparison correction (Bonferroni or Benjamini-Hochberg FDR). Causal verification uses bootstrap confidence intervals, permutation tests, and Cohen's d effect sizes. reptimeline ships three built-in extractors (VQ-VAE, SAE, FSQ), five static and four interactive visualizations, JSON/CSV export, and a command-line interface. We validate on three architecturally distinct backends: a 32-bit binary autoencoder on MNIST (decoder determinism verified: 100%), sparse autoencoder features on Pythia-70M (8/16 features with finite KL selectivity, mean 26.8× L2 ratio with bootstrap 95% CI, plus 8 features with zero cross-activation attributable to SAE sparsity), and a 63-bit neurosymbolic projection head (crystallization event invisible to loss curves). reptimeline is the only tool that combines lifecycle tracking, bottom-up feature discovery, auto-labeling, causal verification, and theory reconciliation in a single backend-agnostic package. Available via `pip install reptimeline` under BUSL-1.1.

1 Introduction

Every system that quantizes neural representations into discrete codes—VQ-VAE codebooks [van den Oord et al., 2017], finite scalar quantization [Mentzer et al., 2024], sparse autoencoders [Cunningham et al., 2023, Sharkey et al., 2022, Bricken et al., 2023], concept bottleneck models [Koh et al., 2020], or binary autoencoders—faces the same blind spot: *you can see what the codes look like after training, but not how they got there.*

Current monitoring tools log scalar aggregates: codebook utilization, perplexity, reconstruction loss. These tell practitioners *that* something changed, not *what* changed. When codebook collapse occurs at step 27,500, a loss curve shows a kink. But which codes died? Did they die together? Was there a phase transition? Which concepts lost their unique representations? These questions require per-code temporal analysis that no existing tool provides.

reptimeline fills this gap. Given a sequence of model checkpoints and a set of probe concepts, it:

1. **Tracks** per-code lifecycle events (births, deaths, connections, phase transitions) and per-bit stability scores;
2. **Discovers** what each code element encodes (duals, dependencies, 3-way interactions) with statistical correction (Bonferroni, BH-FDR);
3. **Labels** discovered features automatically (embedding centroid, contrastive, or LLM);
4. **Verifies** that discovered features are causal via intervention experiments with bootstrap CIs, permutation tests, and effect sizes;
5. **Reconciles** discovered structure against a theoretical ontology and suggests corrections in both directions.

Critically, reptimeline is *backend-agnostic*: it operates on any system that produces a mapping from concepts to discrete code vectors. We validate this claim on three architecturally distinct backends (Section 3).

2 Design and Architecture

2.1 Pipeline Overview

Figure 1 shows the reptimeline pipeline. The user either implements a single abstract class (`RepresentationExtractor`) or uses one of the three built-in extractors (Section 2.3) to load a checkpoint and return discrete codes for a set of probe concepts. Everything downstream is backend-agnostic.

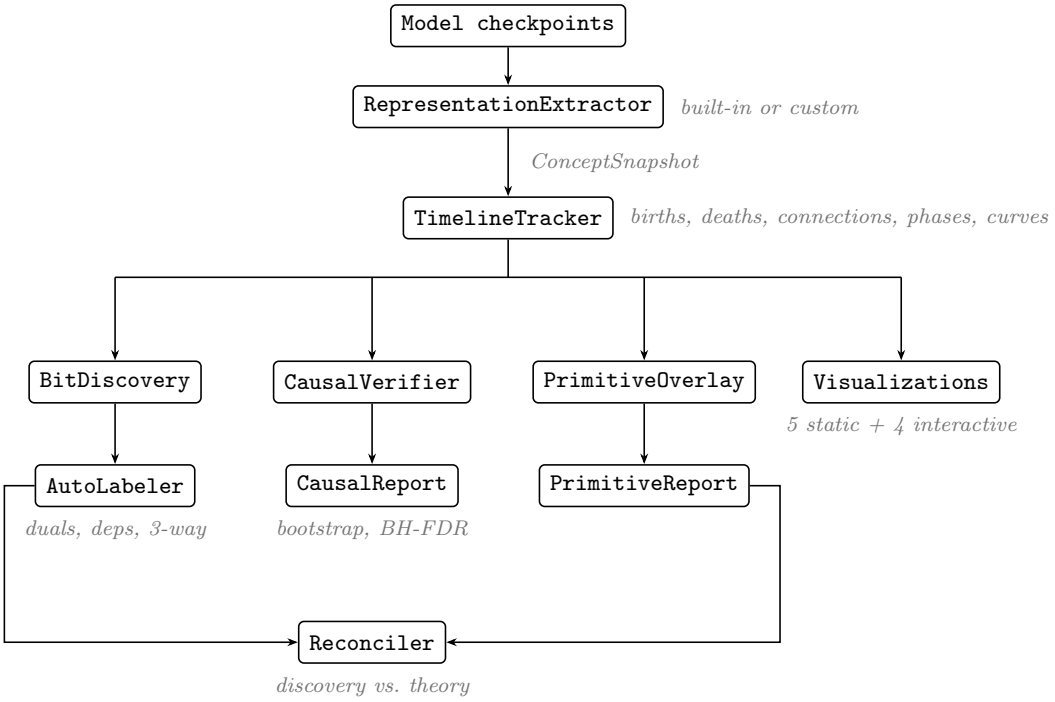

Figure 1: reptimeline pipeline. The user provides only the extractor (or uses a built-in one for VQ-VAE, SAE, or FSQ); all downstream modules are backend-agnostic.

2.2 Core Data Structure

The universal exchange format is `ConceptSnapshot`:

```
ConceptSnapshot(
    step=27500, # training step
    codes={ # concept -> discrete code
        "king": [1, 0, 1, 1, 0, ...],
        "queen": [1, 0, 1, 0, 1, ...],
        ...
    },
    continuous={...}, # optional: continuous embeddings
    metadata={...} # optional: backend-specific info
)
```

This minimal interface—a step number and a dictionary mapping concept names to integer lists—is sufficient to support VQ-VAE codebook indices, SAE binarized activations, binary autoencoder outputs, triadic bit projections, and any future discrete representation scheme. An optional `continuous` field allows hybrid discrete+continuous analysis, and `metadata` stores backend-specific information (e.g., codebook size, SAE top-k).

2.3 Built-in Extractors

While users can implement the `RepresentationExtractor` abstract base class for any model, reptimeline ships three ready-to-use extractors:

- **VQVAEExtractor**: converts VQ-VAE codebook indices into binary indicator vectors (one-hot over the codebook). Similarity via Jaccard index.
- **SAEExtractor**: binarizes sparse autoencoder activations via top-k selection. Includes a built-in intervention helper (`make_intervene_fn`) for zero-ablation causal testing: zero a feature in SAE space, decode back, and measure reconstruction change. Supports optional feature-index mapping for tracking subsets of large dictionaries (e.g., 256 out of 32K features).
- **FSQExtractor**: handles Finite Scalar Quantization [Mentzer et al., 2024] with two binarization modes: *nonzero* (1 if level $\neq 0$, one bit per dimension) and *onehot* (full one-hot per dimension, total bits $= \sum n_{\text{levels}}$). Supports multi-quantizer architectures.

All three extractors inherit automatic checkpoint discovery that parses common naming conventions (`model_step*.pt`, `checkpoint-*.pt`, `step_*.safetensors`) and batch extraction with progress bars.

2.4 Timeline Analysis

`TimelineTracker` consumes a sequence of snapshots and computes:

- **Births**: first step where concept c has bit b active (and it persists).
- **Deaths**: step where bit b becomes permanently inactive for concept c.
- **Connections**: step where concepts c_1, c_2 first share a code element.
- **Curves**: per-step entropy (mean binary entropy across all code elements), churn rate (fraction of concepts whose code changed from the previous step), and utilization (ratio of unique codes to total concepts—closer to 1 means more discriminative).
- **Stability**: per-bit stability scores in $[0, 1]$, measuring the fraction of (concept, step) pairs where a bit remained unchanged from the prior checkpoint.

- **Phase transitions**: detected via statistical change-point analysis—a transition is flagged when $|\Delta_t| > \mu + 2\sigma$ for any curve.

2.5 Feature Discovery

`BitDiscovery` analyzes a single snapshot to discover structure without prior ontological knowledge:

- **Duals**: pairs (b_i, b_j) with Pearson correlation < -0.3 across concepts. These are anti-correlated features—when one is active, the other tends to be inactive.
- **Dependencies**: directed edges $b_p \to b_c$ where $P(b_p = 1 \mid b_c = 1) \geq 0.9$.
- **Three-way AND-gates**: triples (b_i, b_j, b_r) where $P(b_r = 1 \mid b_i = 1 \wedge b_j = 1) \geq \tau$ but $\max(P(b_r \mid b_i), P(b_r \mid b_j)) < \tau$, with $\tau = 0.7$ by default. Additionally, the interaction strength $P(b_r \mid b_i, b_j) - \max(P(b_r \mid b_i), P(b_r \mid b_j))$ must exceed a minimum threshold (default 0.2). These represent features that activate *only* when two other features are jointly active. Complexity is $O(K^3)$ where K is the number of active bits; in practice $K < 100$ and discovery completes in seconds.
- **Hierarchy**: bits ordered by stabilization step—the training step at which a bit's activation pattern stops changing across consecutive checkpoints.

All discovery methods apply multiple-comparison correction: Bonferroni correction for duals and dependencies, and Benjamini-Hochberg FDR correction (via 200-permutation p-values per candidate) for triadic interactions. A `null_baseline()` method generates random codes to estimate false-positive rates under the null hypothesis.

2.6 Auto-Labeling and Reconciliation

`AutoLabeler` translates discovered bit patterns into human-readable labels via three strategies: (1) *embedding centroid*—find the word whose embedding is closest to the centroid of active concepts; (2) *contrastive*—find the word that best separates active from inactive concepts in embedding space (direction vector: $\mathbf{c}_{\text{active}} - \mathbf{c}_{\text{inactive}}$); (3) *LLM*—ask a language model "what do these concepts have in common?" Labels can be exported as a `primitivos.json` file via `export_as_primitives()`, closing the loop from discovery back to overlay analysis.

`Reconciler` compares discovered structure against a theoretical ontology (`PrimitiveOverlay`) and classifies mismatches by type and severity:

- **Bit mismatches**: *dead_but_assigned* (critical—primitive's bit never activates), *active_but_unassigned* (warning—active bit has no primitive), *semantic_drift* (info—bit too broadly active).
- **Dual mismatches**: anti-correlated pairs present in theory but absent in model (or vice versa).
- **Dependency mismatches**: required parent→child edges absent in model or discovered edges absent in theory.

An *agreement score* quantifies overall alignment: $1 - (\text{critical mismatches}/n_{\text{primitives}})$. The Reconciler produces correction suggestions in *both directions*: to the model's training anchors (how to retrain) and to the theory (how to update `primitivos.json`).

2.7 Causal Verification

`CausalVerifier` tests whether discovered features have selective causal effects. The user provides an intervention function $f(c, b) \to \mathbb{R}$ that returns the effect of ablating bit b for concept c (e.g., L2 reconstruction change or KL divergence). For each tested bit:

1. Concepts are partitioned into *labeled* (bit = 1) and *other* (bit = 0) groups.
2. The intervention function is called for all concepts.

3. A **bootstrap confidence interval** (default 1,000 resamples) is computed on the selectivity ratio: $\bar{f}_{\text{labeled}}/\bar{f}_{\text{other}}$.
4. A two-sided **permutation test** (default 1,000 permutations) yields a p-value.
5. **Cohen's** d measures effect size between groups.
6. **Benjamini-Hochberg FDR correction** is applied across all tested bits at $\alpha = 0.05$.

A bit is considered causally selective if it is significant after FDR correction *and* its selectivity ratio exceeds a threshold (default 1.5×). The overall verdict is *causal_evidence_found* if at least 3 bits meet both criteria. All statistical functions (`bootstrap_ci`, `permutation_test`, `benjamini_hochberg`, `effect_size_cohens_d`) are implemented in `stats.py` using only NumPy, with no SciPy dependency.

2.8 Visualization and CLI

reptimeline provides five static visualizations (matplotlib):

1. **Phase dashboard**: three-panel plot of entropy, churn, and utilization curves with phase transitions marked as vertical lines.
2. **Swimlane**: per-concept heatmap where rows are concepts, columns are training steps, and cells show bit activation state (Figure 4).
3. **Churn heatmap**: per-bit churn rates across training, highlighting the least stable bits.
4. **Layer emergence**: horizontal bar chart showing when each layer's primitives first stabilize.
5. **Causal heatmap**: selectivity ratio per bit with bootstrap confidence intervals as error bars.

Four of these (phase dashboard, swimlane, causal heatmap, churn heatmap) also have interactive Plotly versions with hover details and zoom, available when Plotly is installed.

Export. `Timeline` supports `save_json()` / `load_json()` for persistence and `to_csv()` for exporting five CSV files: events, connections, curves, stability, and codes.

CLI. The command-line interface provides access to the full pipeline:

```
reptimeline --snapshots data.json --discover --plot
reptimeline --snapshots data.json --overlay primitivos.json
reptimeline --snapshots data.json --causal effects.json --output results.json
```

3 Case Studies

We validate reptimeline on three backends spanning supervised image models, unsupervised LLM internals, and end-to-end neurosymbolic systems. Table 1 summarizes the results.

3.1 MNIST Binary Autoencoder

Setup. A binary autoencoder with a 32-bit bottleneck using straight-through estimation [Bengio et al., 2013] was trained on MNIST for 10 epochs. reptimeline extracted majority-vote codes for each of the 10 digit classes at 6 checkpoints.

Lifecycle. reptimeline detected 297 bit births and 106 deaths across training, with 45 concept-pair connections forming as digits developed shared features. 31 of 32 bits were active; 1 bit was permanently dead.

Discovery. BitDiscovery found 65 anti-correlated dual pairs and 179 dependency edges, revealing the internal structure the autoencoder learned to separate digit classes.

Causal verification. A *code swap* experiment tested whether the 32-bit code causally determines the output: for each pair of digits (d_i, d_j), we encoded d_i's image but decoded using

Table 1: reptimeline analysis across three backends. All numbers are from production runs, not toy examples.

	MNIST BAE	**Pythia SAE**	**TriadicGPT**
Architecture	Binary AE (32-bit)	SAE on Pythia-70M	Triadic head (63-bit)
Concepts	10 digits	60 (6 domains)	53
Code dimension	32	256 (from 32K)	63
Checkpoints	6	12	8
Births	297	1,835	2,632
Deaths	106	875	1,732
Connections	45	1,770	1,378
Phase transitions	0	2	3
Duals discovered	65	34	9
Dependencies	179	227	—
Triadic (3-way)	—	74	—
Causal evidence	**100% decoder det.**	**8/16 finite sel.**	**Crystallization**

d_j's binary code. The decoder produced d_j's reconstruction in 100% of cases ($n = 100$ swaps), confirming decoder determinism: the 32-bit code fully determines the output. This verifies the code→output mapping, not the input→code mapping (Figure 2).

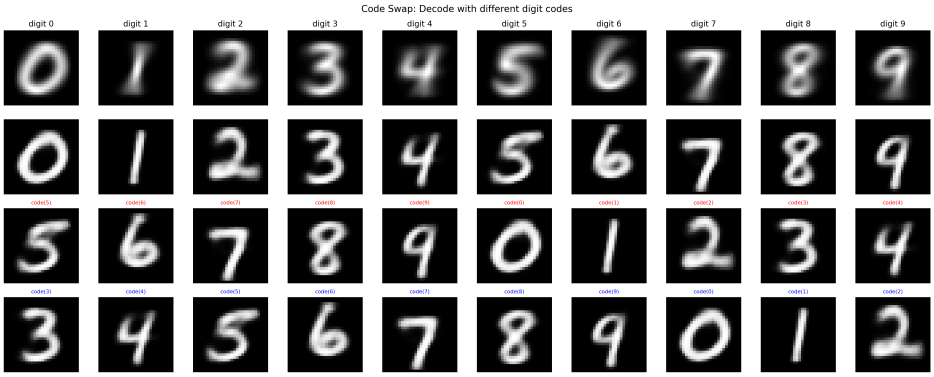

Figure 2: MNIST code swap causality test. Decoding with digit X's 32-bit code always produces digit X's reconstruction, regardless of the input image. The discrete code fully determines the output.

3.2 Pythia-70M Sparse Autoencoder

Setup. We used EleutherAI's pre-trained sparse autoencoder (`sae-pythia-70m-32k`) on layer 3 MLP outputs of Pythia-70M [Biderman et al., 2023]. We defined 60 probe concepts across 6 semantic domains (physical, animals, emotions, abstract, social, properties) and tracked 256 SAE features (selected by activation frequency at the final checkpoint) across 12 training checkpoints from step 1 to step 143,000.

Lifecycle. reptimeline detected 1,835 births, 875 deaths, and 1,770 connections, with 2 phase transitions detected automatically. 64 of 256 tracked features were active (the remaining 192 were effectively dead—expected given SAE sparsity).

Discovery. BitDiscovery found 34 dual pairs, 227 dependency edges, and 74 three-way AND-gate interactions. Auto-labeling via embedding centroids produced interpretable names: "teacher," "belief," "dog," "wisdom," "ocean." Figure 4 shows the swimlane heatmap for the first 20 features across 12 checkpoints.

Causal verification. We tested all 16 active features via *SAE causal intervention*: for each feature, we encoded all 60 concepts' hidden states via the SAE, zeroed the target feature, decoded back to hidden-state space, and injected the modified state into the model via a forward hook at layer 3. We measured the KL divergence between original and modified next-token predictions.

8/16 features showed **finite KL selectivity** averaging 26.8× L2 (best: 98.4× for "fast"; "teacher" at 26.6× disproportionately affected *child, dog, wolf, queen*). The remaining 8 features showed zero cross-activation, which may reflect SAE sparsity (the feature never fires for non-labeled concepts) rather than proven causal selectivity; we report these groups separately. L1-norm ratios yield a lower mean of 19.5× on the same features. Figure 3 shows the selectivity heatmap.

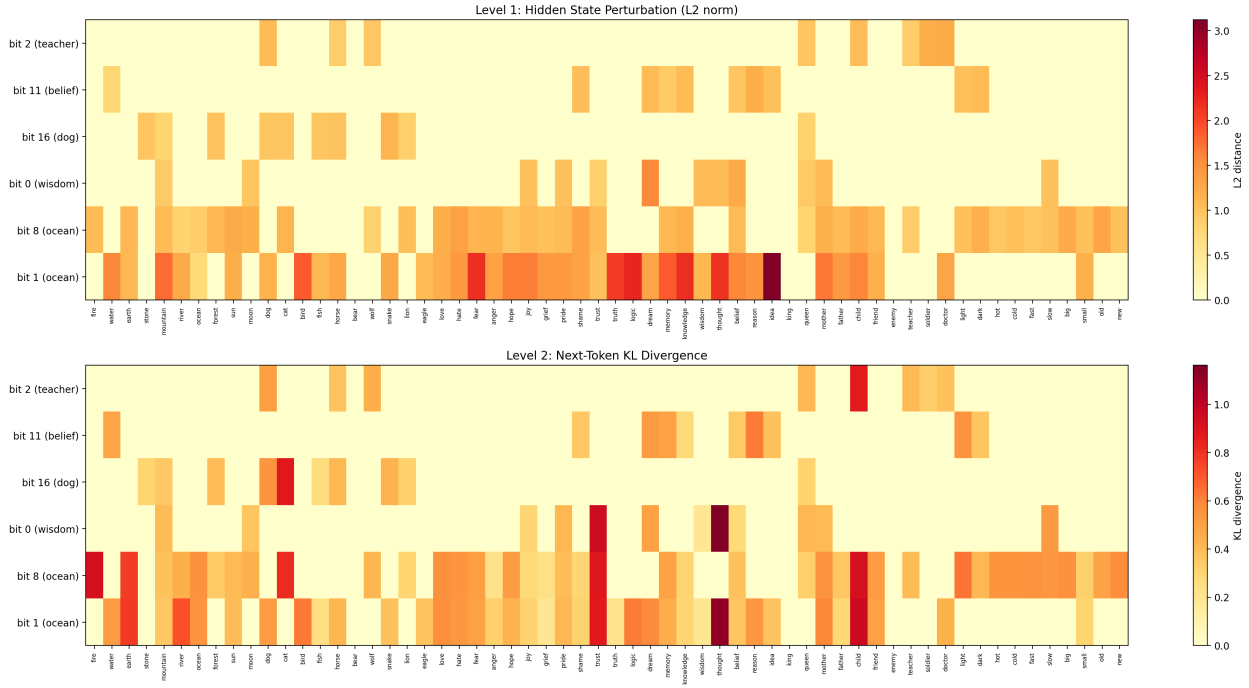

Figure 3: Causal intervention on Pythia-70M SAE features (16 tested). **Top**: hidden state perturbation (L2 norm). **Bottom**: KL divergence in next-token predictions. Eight features are perfectly selective (only affect labeled concepts); the remaining eight average 26.8× selectivity. "fast" (bit 14): 98.4×; "teacher" (bit 2): 26.6×. Yellow = no effect; dark red = strong effect.

3.3 TriadicGPT (63-bit Neurosymbolic Head)

Setup. TriadicGPT [Ornelas Brand, 2026b] is a 40M-parameter GPT that jointly learns next-token prediction and 63-bit discrete semantic projections via a triadic head trained end-to-end. reptimeline tracked 53 concepts across 8 checkpoints from step 2,500 to step 50,000.

Lifecycle and phase discovery. reptimeline revealed a *four-phase training dynamic* invisible to standard scalar logging:

1. **Exploration** (steps 0–7,500): high bit churn across all concepts.

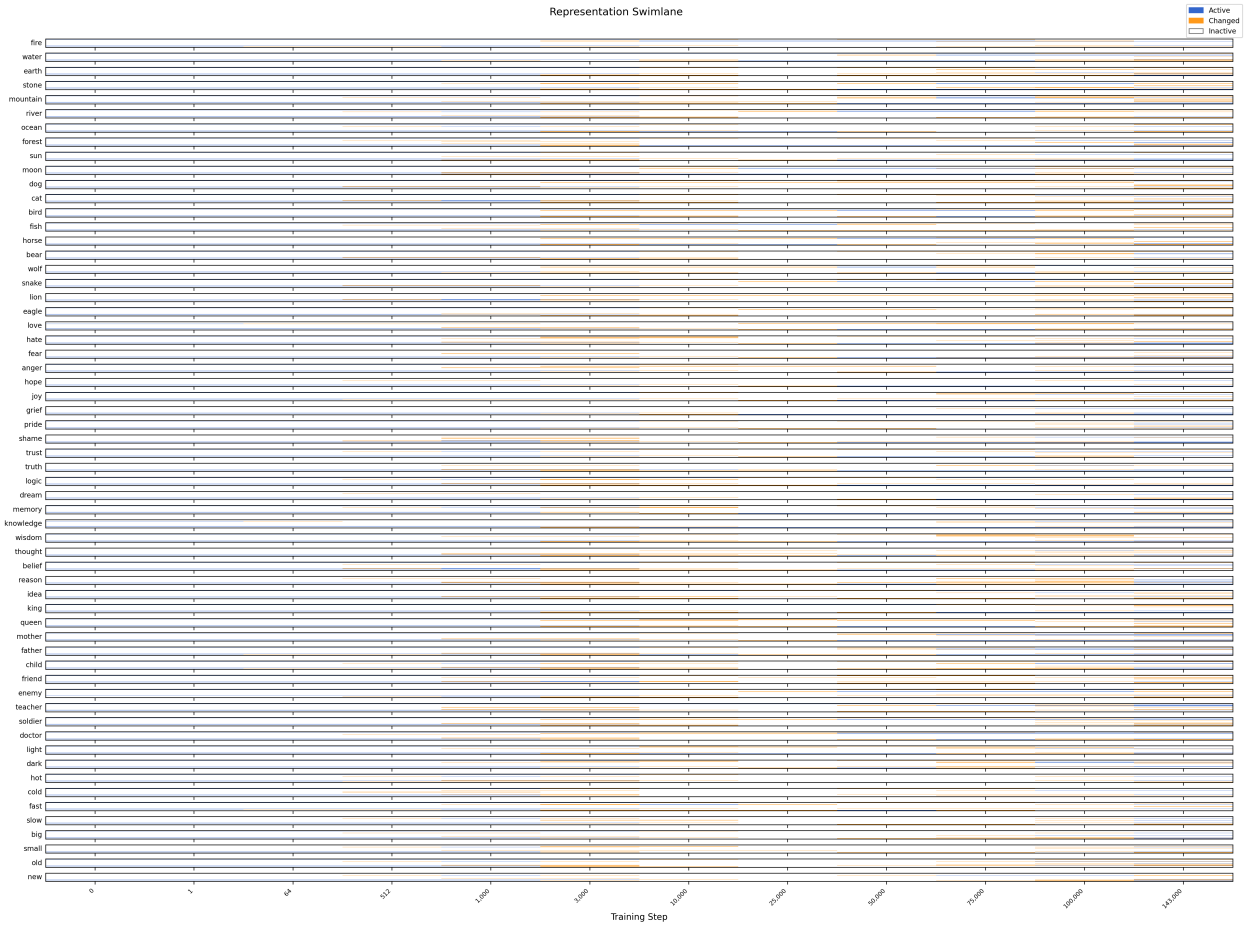

Figure 4: Swimlane visualization for Pythia-70M SAE features. Each row is a probe concept, each column a training checkpoint. Cell color indicates bit activation state: blue = active, white = inactive, orange = changed from previous step. The visualization reveals which concepts acquire or lose features over training and how representation structure crystallizes.

2. **Consolidation** (7,500–27,500): churn remains elevated but entropy stabilizes.
3. **Crystallization** (∼27,500): entropy drops from 0.50 to 0.23 and utilization drops from 0.98 to 0.66 simultaneously—a sharp phase transition.
4. **Stabilization** (27,500–50,000): churn collapses to 0.075; representations freeze.

The crystallization event—the moment the model "commits" to its semantic assignments—is not visible in loss curves or standard metrics. Related crystallization phenomena have been studied in grokking [Power et al., 2022, Nanda et al., 2023] and VQ-VAE codebook dynamics, but to our knowledge reptimeline is the first tool to detect such transitions automatically via per-code temporal analysis.

Structural causality. Unlike MNIST and Pythia where features lie in the computation path, TriadicGPT's triadic head is a parallel projection—flipping a bit direction in hidden space changes LM output (mean KL > 0) but not selectively (1/24 bits above 1.5×). This is an expected architectural consequence, not a tool limitation. The causality evidence for TriadicGPT is *structural*: reptimeline discovers 9 anti-correlated dual pairs (e.g., vida/muerte at $r = 0.91$, placer/dolor at $r = 1.00$) that match the gold semantic axes defined in the training ontology, confirming that the model learned the intended semantic structure.

Overlay analysis. The `PrimitiveOverlay` module loaded the manually-defined ontology specifying 63 named primitives with layer assignments, dependency edges, and dual pairs. It computed four diagnostics: (i) *activation epochs*—the first step each primitive activates per concept; (ii) *dependency completions*—when all required dependencies are simultaneously active; (iii) *layer emergence*—aggregate statistics per layer (earliest, median, latest activation step); (iv) *dual coherence*—whether anti-correlated pairs exhibit exclusive activation, scored as $n_{\text{excl}}/(n_{\text{excl}}+n_{\text{both}})$. Layer emergence confirmed that lower-layer primitives (structural) stabilized before higher-layer ones (semantic), consistent with the expected hierarchy.

Reconciliation. The Reconciler achieved 44.4% agreement between model-discovered and manually-defined ontology, identifying 35 critical mismatches (including 8 *dead_but_assigned* and 12 *active_but_unassigned* bits) that guided anchor expansion from 50 to 158 concepts in subsequent training iterations. The Reconciler produced correction suggestions for both directions: 5 anchor retraining recommendations and 8 theory update suggestions.

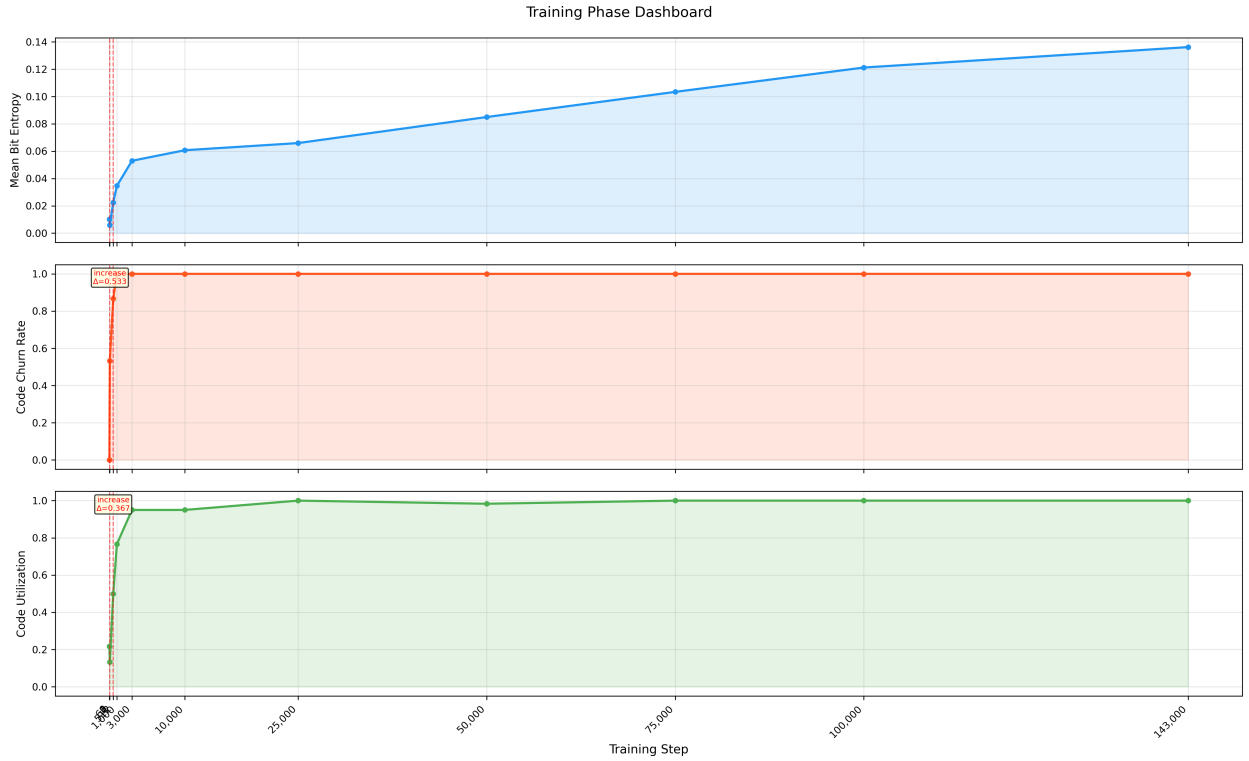

Figure 5: Pythia-70M phase dashboard: entropy, churn rate, and utilization curves across 12 training checkpoints. Vertical lines mark automatically detected phase transitions. Similar four-phase dynamics were observed in TriadicGPT training.

3.4 Prediction Experiment: Negative Results

We tested whether the 8 finite-selectivity SAE features could improve next-token prediction. Two strategies were evaluated: (1) embedding-based feature selection from the discovered set; (2) MLP-based prediction using feature activations. Neither improved over the unmodified Pythia-70M baseline: embedding-based achieved -0.13% relative accuracy, and MLP-based achieved -4.20%. The individual features are causally meaningful (removing them selectively changes predictions)

but combining them for prediction does not yet yield improvements. This suggests that selective causal influence is necessary but not sufficient for useful feature-based prediction.

4 Comparison with Existing Tools

Table 2 compares reptimeline with existing approaches for monitoring and interpreting discrete representations.

Table 2: Feature comparison across discrete representation tools. ✓ = full support, Partial = aggregate only, Manual = requires human effort, — = not supported.

Capability	**reptimeline**	VQ-VAE	SAE	CBM	RepE
Per-code lifecycle	✓	Partial	Partial	—	—
Connection tracking	✓	—	—	—	—
Phase transition detection	✓	—	Manual	—	—
3-way interaction discovery	✓	—	—	—	—
Bottom-up ontology discovery	✓	—	—	—	—
Auto-labeling (3 strategies)	✓	—	Manual	Manual	—
Causal verification (stat.)	✓	—	—	—	✓
Multiple-comparison correction	✓	—	—	—	—
Built-in extractors	✓ (3)	—	—	—	—
JSON/CSV export	✓	—	Partial	—	—
Backend-agnostic	✓	—	—	—	—

VQ-VAE tools (e.g., WandB codebook metrics) track aggregate utilization and commitment loss but not per-code lifecycle events. **SAE dashboards** (e.g., Neuronpedia [Bricken et al., 2023, Templeton et al., 2024]) provide manual feature browsing for sparse autoencoders but are architecture-specific and do not track temporal evolution. Recent work on automated circuit discovery [Conmy et al., 2023] identifies computational circuits but does not track discrete code evolution. The superposition hypothesis [Elhage et al., 2022] motivates SAE-based feature extraction; reptimeline operates downstream, analyzing the features SAEs produce. **Concept Bottleneck Model (CBM) tools** [Koh et al., 2020] require predefined concept annotations and do not discover structure bottom-up. **Representation Engineering (RepE)** [Zou et al., 2023] identifies directions in continuous space but does not operate on discrete codes.

reptimeline differs fundamentally: it operates on the discrete codes themselves, requires no predefined labels, discovers structure bottom-up, and verifies causality—all through a single backend-agnostic API.

Note: Table 2 compares tool categories rather than specific implementations. Individual tools within each category may offer partial overlapping capabilities not captured by a binary comparison.

5 Conclusion and Availability

reptimeline fills a gap in the discrete representation toolbox. It reveals training dynamics that scalar logging misses (crystallization events), discovers interpretable structure without supervision (duals, dependencies, three-way interactions), and verifies that discovered features are causally meaningful—with rigorous statistical methodology including bootstrap confidence intervals, permutation tests, Benjamini-Hochberg FDR correction, and Cohen's d effect sizes (8/16 with finite KL selectivity at 26.8× mean on Pythia-70M, 100% decoder determinism on MNIST).

Availability. `pip install reptimeline`. Source code, examples, and pre-computed results are available at https://github.com/arturoornelasb/reptimeline (software DOI: 10.5281/zenodo.19208628). Licensed under BUSL-1.1 (free for research; converts to AGPL-3.0 in 2030). The package includes 224 unit tests at 92% coverage, 11 example pipelines, 6 visualization modules (5 static, 4 interactive), 3 built-in extractors (VQ-VAE, SAE, FSQ), JSON/CSV export, a command-line interface, and a structured exception hierarchy. All statistical methods are implemented in pure NumPy with no SciPy dependency.

Future work. (1) Streaming mode for real-time monitoring during training. (2) Integration with WandB/TensorBoard as a plugin. (3) Extension to continuous representations via learned discretization. (4) Active learning loop: use Reconciler mismatches to automatically suggest retraining targets.

References

Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *ICML*, 2023.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

P. W. Koh, T. Nguyen, Y. S. Tang, S. Mussmann, E. Pierson, B. Kim, and P. Liang. Concept bottleneck models. In *ICML*, 2020.

F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen. Finite scalar quantization: VQ-VAE made simple. In *ICLR*, 2024.

A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. In *ICLR Workshop*, 2022.

J. A. Ornelas Brand. Triadic Neurosymbolic Engine: Prime factorization as a neurosymbolic bridge: Projecting continuous embeddings into discrete algebraic space for deterministic verification. 2026. doi:10.5281/zenodo.19205805.

J. A. Ornelas Brand. End-to-end prime factorization in a generative language model: Emergent algebraic semantics from joint training. 2026. doi:10.5281/zenodo.19206545.

A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *NeurIPS*, 2017.

A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A. Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. In *ICLR*, 2023.

T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, Anthropic, 2023.

L. Sharkey, D. Braun, and B. Millidge. Taking features out of superposition with sparse autoencoders. *arXiv preprint arXiv:2211.11279*, 2022.

A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *NeurIPS*, 2023.

N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, Anthropic, 2022.

A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, Anthropic, 2024.